

Development Process Report: Multi-Schema Rendering Engine

1. Executive Summary

The project evolved from a single-schema utility into a Schema-Agnostic Rendering Engine. The architecture now supports diverse healthcare XML documents (Patient, Encounter, Billing) through a dynamic registry-based system. The primary design goals were to eliminate hardcoded logic, ensure schema-level isolation, and provide a seamless web-based user experience for document conversion.

2. System Architecture

The system maintains a three-tier decoupled architecture:

- **Ingestion Layer (Dynamic Registry):** Uses a Strategy Pattern to register schema-specific parsers.
- **Data Modeling Layer:** Uses Pydantic models to act as a "Canonical Data Store." Regardless of the source XML, data is normalized into these models before rendering, ensuring type safety.
- **Presentation Layer:** A web-based UI (FastAPI) with dynamic template lookups, converting normalized data models into audit-ready PDF reports via pdfkit.

3. Design Decisions & Technical Justification

Why DOM vs. SAX/StAX?

I selected DOM (Document Object Model) parsing with lxml.

- **Reasoning:** DOM enables XPath expressions, essential for navigating the deeply nested, hierarchical structures of clinical XML.

- **Contextual Integrity:** DOM allows for bidirectional node lookups, which are critical when data must be aggregated from varied document sections. Given that document modules are discrete units of information, the memory footprint of a DOM tree is optimal compared to the complexity overhead of event-driven stream parsing.

PDF Library Selection

I selected pdfkit (wrapping wkhtmltopdf).

- **Reasoning:** This enables an HTML/CSS-to-PDF pipeline, creating a strict separation between Content and Presentation.
- **Regulatory Alignment:** This approach allows for rapid changes to branding, headers, and clinical layouts via CSS, satisfying requirements for highly regulated industries where the presentation of data often changes more frequently than the data itself.

XML-to-Data Mapping Structure

The mapping follows a Three-Tiered Strategy:

1. **Extraction:** Explicit XPath queries (namespace-aware) extract raw data.
2. **Normalization:** Raw XML is serialized into strongly-typed Pydantic Data Models. This layer enforces data integrity—if the XML fails to meet the schema, the system raises a validation error immediately.
3. **Encapsulation:** The logic is encapsulated within specific Parser classes (implementing a BaseParser interface), ensuring the rendering engine is never exposed to raw XML, only to the validated models.

4. Answers to Level 2 Design Questions

How does your schema-to-template mapping work?

I implemented a Registry Pattern. A SCHEMA_REGISTRY dictionary acts as the "Single Source of Truth." It maps an identified schema_type (e.g., "PATIENT") to a specific Parser class and a corresponding Jinja2 template file. This allows the system to be "Open for Extension": adding a new schema requires only creating a parser class and adding one entry to the registry dictionary—zero modifications to the core engine are required.

How do you distinguish between schemas on upload?

I implemented **XSD-Based Schema Detection**. Rather than relying on fragile filename conventions or user input, the system:

- **Validates against registered XSD definitions:** The uploaded XML is validated against the actual XSD definition file for each registered schema using lxml's built-in XSD validator
- **Deterministic schema matching:** Tests the XML sequentially against all available XSD files and returns the matching schema immediately upon successful validation
- **Strict data integrity enforcement:** Rejects files that don't validate against any registered XSD, ensuring only compliant, well-formed XML enters the pipeline

This approach prioritizes **correctness over leniency**—a critical requirement for healthcare systems. By relying solely on XSD validation, the system guarantees that all accepted data strictly conforms to its declared schema, eliminating ambiguity and preventing malformed data from causing downstream issues during parsing or rendering.

What abstraction layer sits between parsed XML data and the template rendering step?

The Pydantic Model Layer acts as the primary abstraction. This model layer is the "Canonical Data Store." By passing the Pydantic object (the p object in the template) to the renderer, we ensure the template logic is completely agnostic to the source XML structure. The template engine only interacts with clean, validated Python objects, which makes the templates highly reusable and easy to test.

5. Development Challenges & Scalability

- **Concurrency:** To prevent file collision in a multi-user environment, I

implemented a uuid-based temporary file management system, with BackgroundTasks to ensure non-blocking cleanup of generated files.

- **Schema Variance:** Handling highly complex schemas (like Medical Billing) required moving from simple list iteration to recursive object mapping to maintain clinical data integrity (e.g., correctly linking diagnoses to specific service lines).

6. Conclusion

The combination of Dynamic Schema Detection, the Strategy Pattern, and Strict Type Validation creates a system that is both extensible and audit-resilient, fitting the high-standard requirements of the healthcare documentation sector.